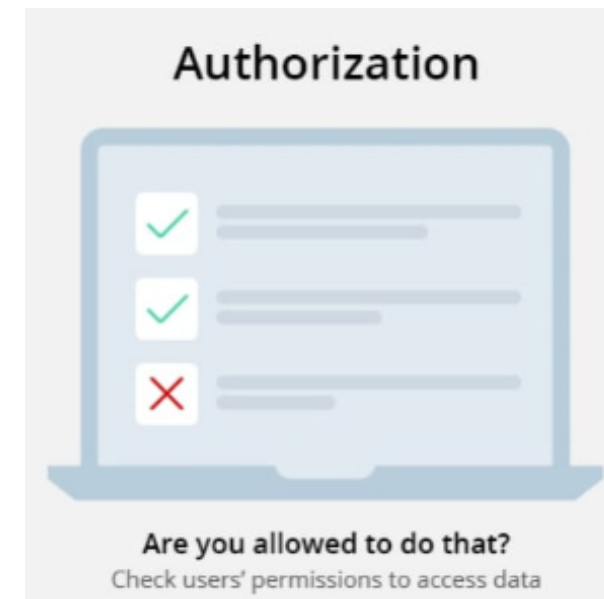
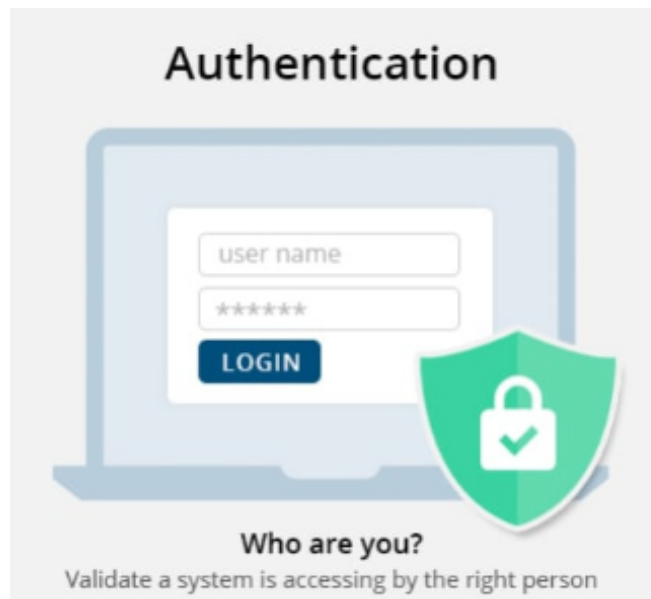


Week02_Authentication

Authentication이 무엇인가 부터 1day 취약점 분석까지

Authentication vs Authorization

- ▶ 인증(Authentication)과 인가(Authorization)

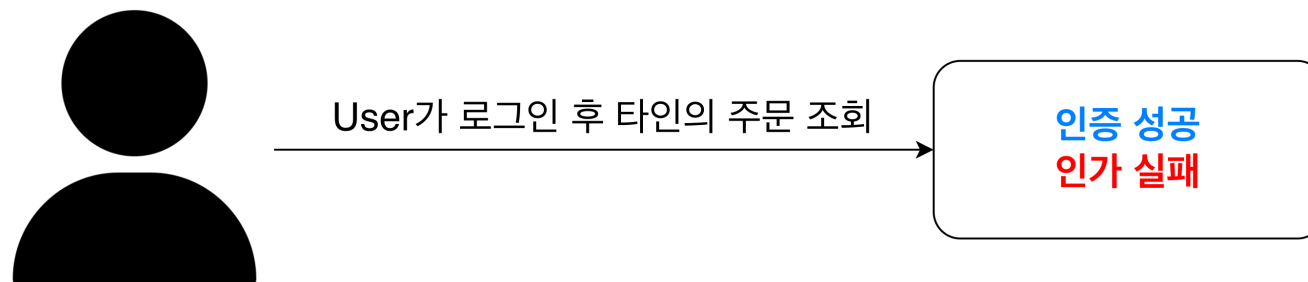
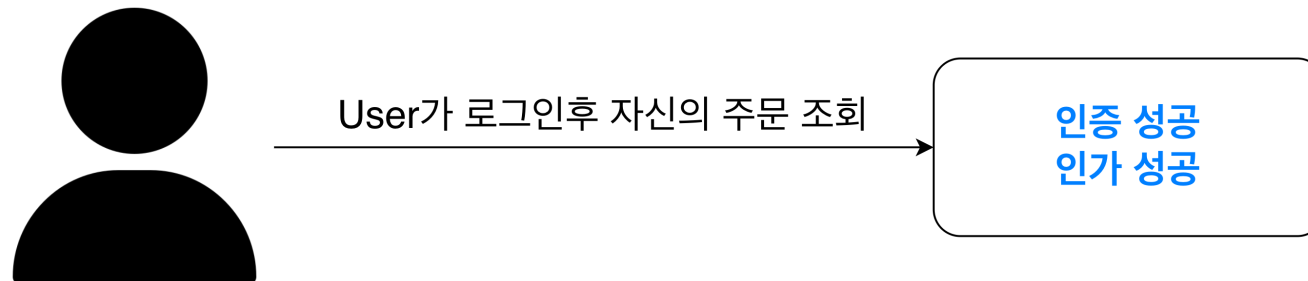
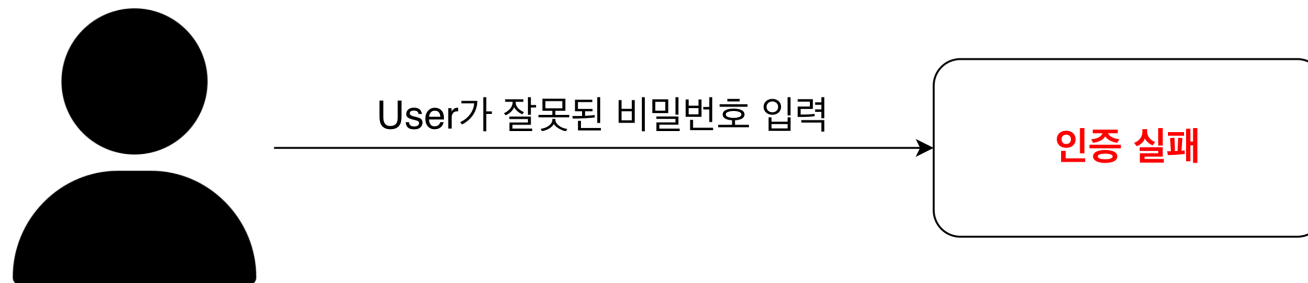


- 인증: 사용자가 주장하는 신원을 확인하는 과정
 - 지식 요소: 내가 아는 것 (비밀번호)
 - 소유 요소: 내가 가지고 있는 것 (보안 키)
 - 생체 및 행동 요소: 나의 특성 (지문, 얼굴)
- 인가: 권한을 확인하는 과정

즉, 인증은 누구인가 를 확인하는 과정이고 인가는 이 작업을 해도 되는가를 확인하는 과정이다.

Example. 쇼핑몰에서의 인증과 인가

- ▶ 일반 회원은 자신의 주문만 조회할 수 있다 가정



인증 취약점의 원인과 영향

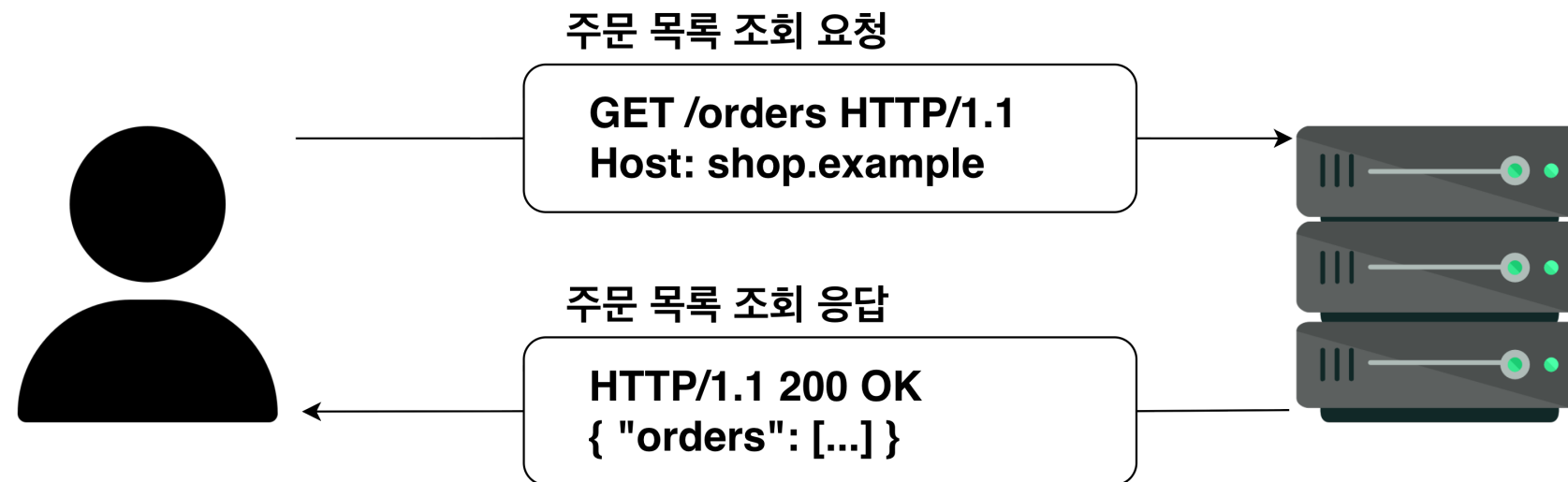
- ▶ 인증 취약점이 발생하는 대표적인 원인
 - 무차별 대입 공격에 대한 적절한 방어 기능을 제공하지 못함
 - 로그인 추측 시도를 제한하지 않음
 - 구현 오류로 인해 필요한 검증을 건너뛸 수 있음
- ▶ 만일 다른 사람으로 인증되면 그 계정의 권한으로 정보와 기능에 접근할 수 있다.



즉, 인증 취약점의 피해는 해당 계정이 가진 권한과 연결된다.

HTTP 요청과 REST API

- ▶ REST API: HTTP 메서드와 리소스 경로로 서버의 데이터나 기능을 요청하는 방식



- HTTP 메서드: 요청할 작업
 - GET: 조회 / POST 생성 / PATCH 수정 / DELETE 삭제
- 리소스 경로: 요청할 데이터나 기능의 위치
 - Eg. /api/orders
- API 엔드포인트: 서버나 API 요청을 받도록 제공하는 주소
 - Eg. GET /api/orders

보호된 API에서는 서버가 요청자의 인증 정보와 작업 권한을 확인해야 한다.

요청에 담기는 인증 정보

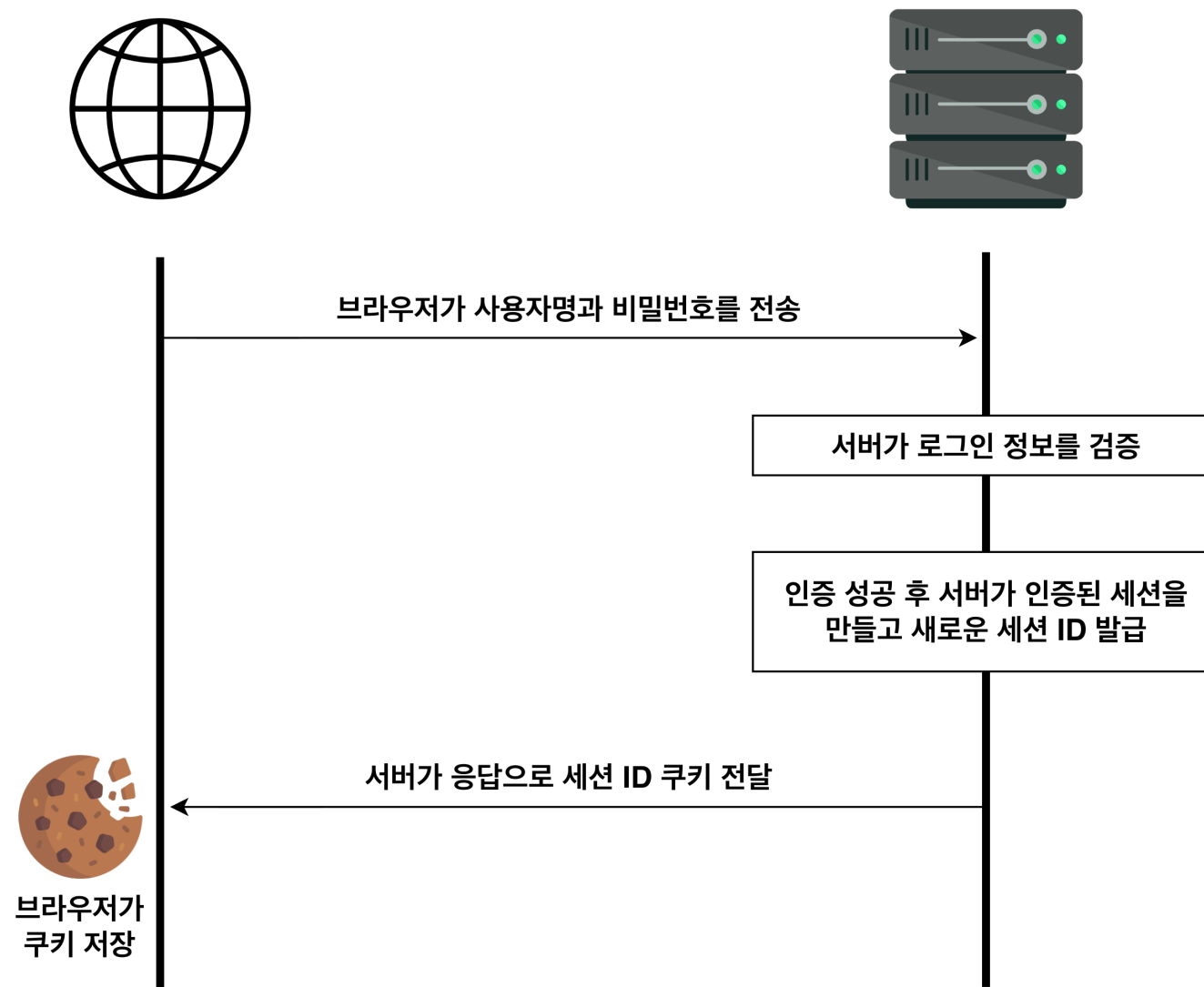
- ▶ 세션 ID를 쿠키로 전달하는 예시
 - 브라우저가 쿠키에 저장
 - 이후 요청에 자동으로 쿠키 첨부
 - 서버가 전달받은 세션 ID로 사용자 상태 조회
- ▶ Bearer 토큰을 헤더로 전달하는 예시
 - 클라이언트가 요청 헤더에 첨부
 - 서버가 토큰의 유효성과 연결된 사용자 확인
 - API 인증에서 자주 사용
- Bearer
 - 유효한 토큰을 가지고 제시한 사용자를 해당 토큰의 사용자로 처리하는 방식

Cookie: session=<세션 ID>

Authorization: Bearer <인증 토큰>

```
GET /orders HTTP/1.1
Host: shop.example
Authorization: Bearer abc123
```

세션, 세션 ID, 쿠키



- 세션(Session): 요청 사이에 유지하는 사용자 관련 상태
- 세션 ID: 특정 세션을 식별하는 값
- 쿠키(Cookie): 브라우저가 저장하고 요청에 첨부하는 작은 데이터

보호된 API의 인증 검사

- ▶ 보호된 기능을 실행하기 전에 확인할 사항
 1. 필요한 인증 정보가 있는가?
 2. 인증 정보가 유효한가?
 3. 어떤 사용자의 요청인가?
- 인증 정보가 없거나 유효하지 않음 → 요청 거부
- 인증 성공 → 해당 기능에 대한 권한 확인

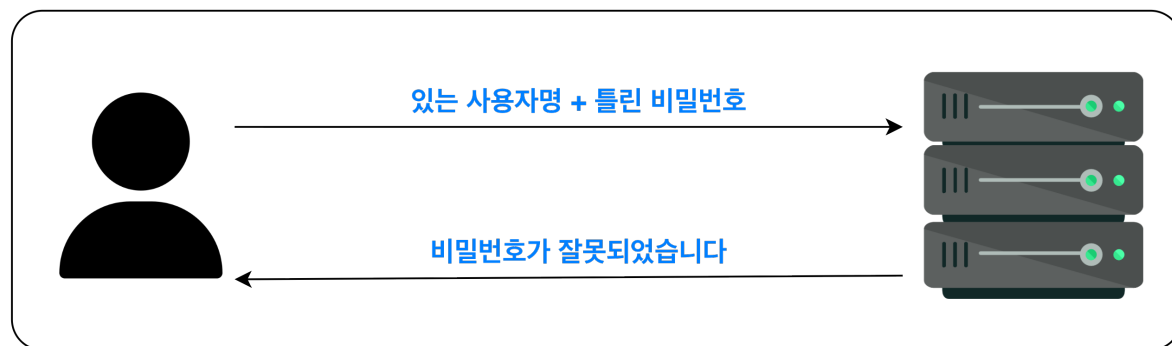
인증과 권한 확인을 통과한 요청만 보호된 기능을 실행한다.

Authentication Vulnerabilities

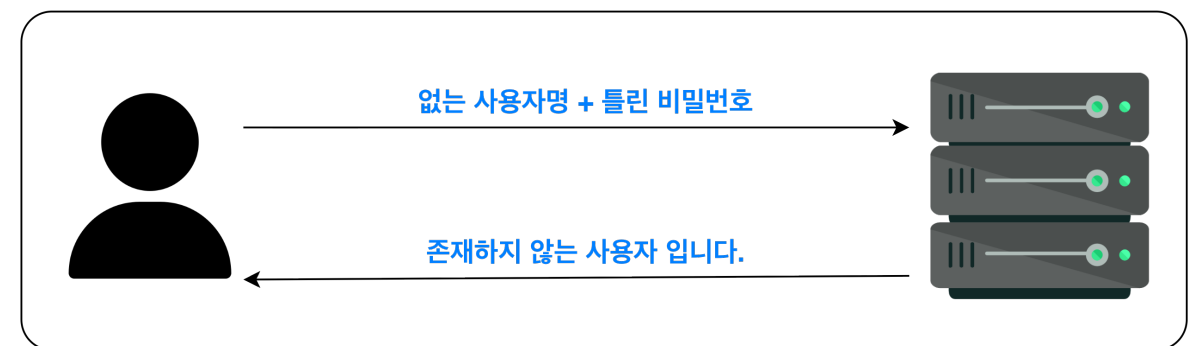
비밀번호 기반 인증의 취약점

- ▶ 서버는 사용자명과 비밀번호를 검증해 인증 여부를 결정한다.
- ▶ 이 과정에서 확인할 문제
 - 로그인 실패 응답이 계정의 존재를 알려주는가?
 - 비밀번호를 계속 추측할 수 있는가?
 - 반복적인 로그인 시도를 실제로 제한하는가?
- ▶ 사용자명 열거(Username Enumeration)
 - 서비스의 반응 차이로 존재하는 계정을 식별

계정이 존재할 때



계정이 존재하지 않을때

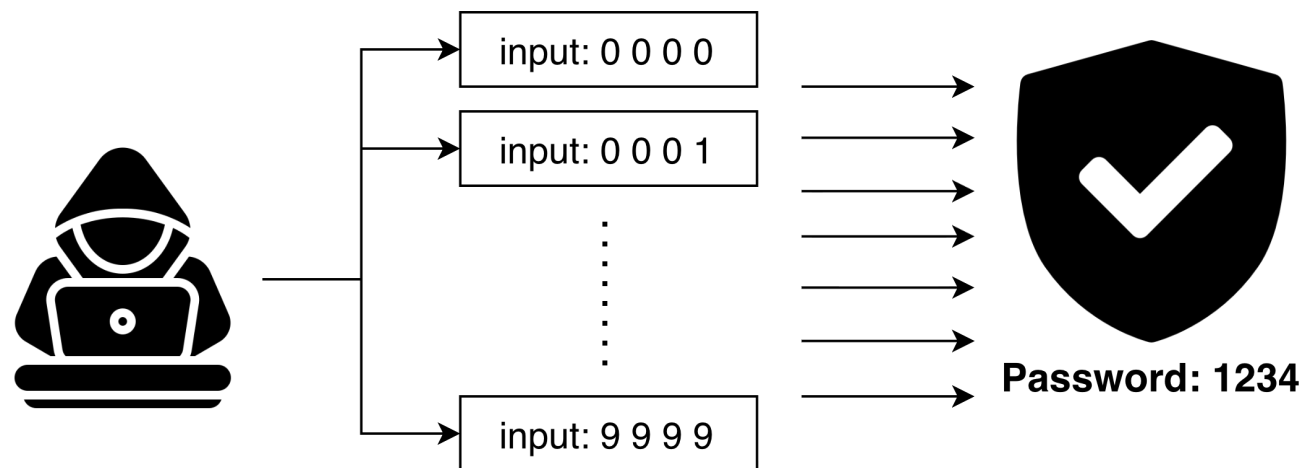


- 두 요청은 모두 인증에 실패했지만, 계정의 존재 여부가 드러난다.

비밀번호 기반 인증의 취약점

▶ 무차별 대입 (Brute-force)

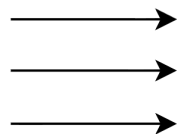
- 로그인 정보를 반복해서 추측하는 공격



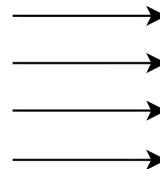
- 자동화 시 많은 후보 빠르게 시도 가능
- 흔하거나 예측 가능한 비밀번호는 추측에 취약

- 로그인 시도 제한등의 기법으로 반복적인 추측의 횟수와 속도를 줄일 수 있다.
 - 공격을 늦추면서 정상 사용자의 이용도 고려해야 하는 Trade-off가 존재

Too Many Attempts



192.168.X.X



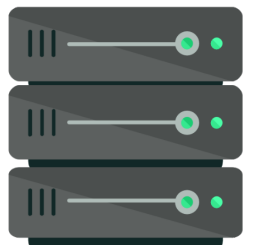
Block 192.168.X.X



Repeated Failure

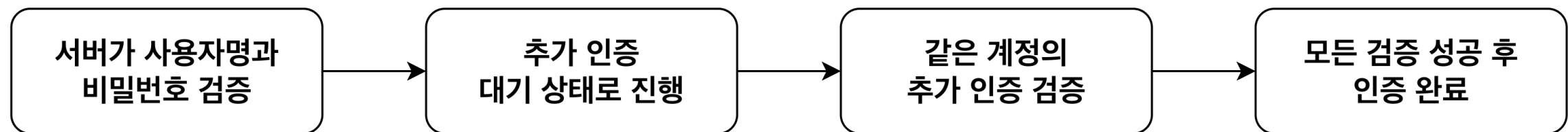


Increased Waiting Time



다중 요소 인증

- ▶ 2FA (Two-Factor Authentication): 서로 다른 두 종류의 인증 요소를 확인하는 방식
 - Eg. 비밀번호 검증 + 인증 앱의 일회용 코드 검증 (전남대학교의 현재 방식)
 - 2FA의 정상적인 인증 흐름

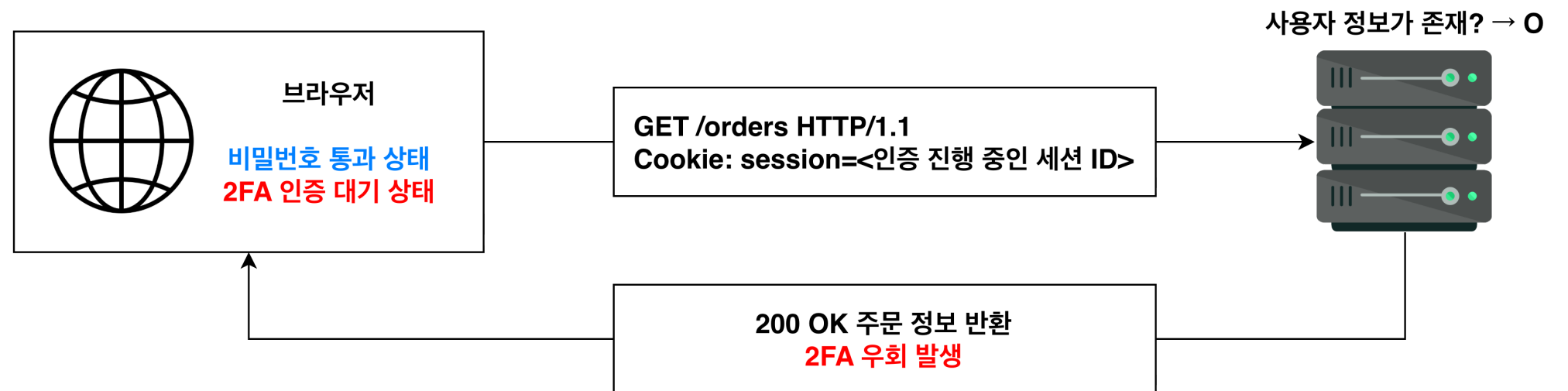


- 2FA가 요구하는 모든 인증을 통과해야 인증 완료

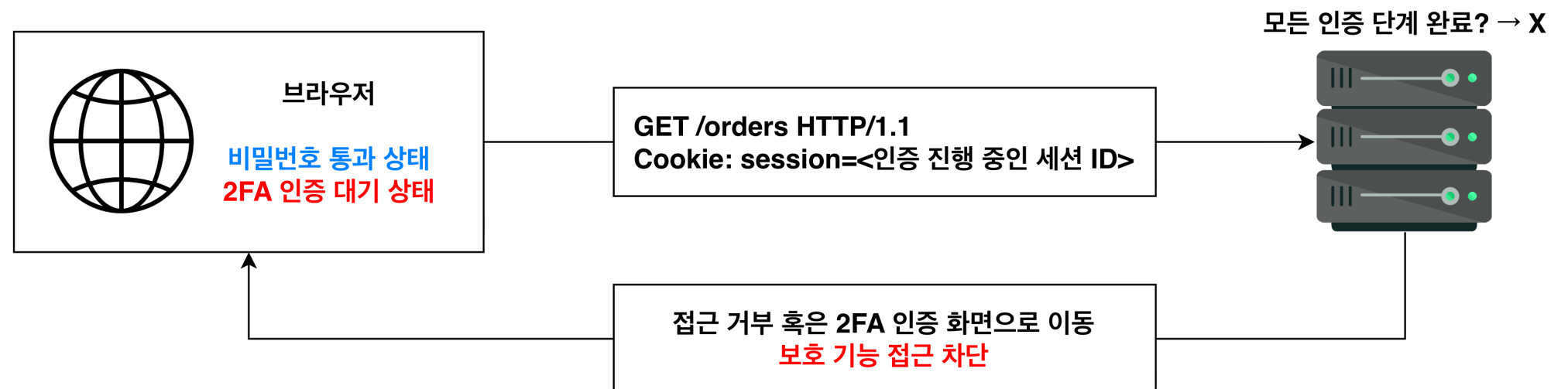
다중 요소 인증

- ▶ 비밀번호 검증 후, 추가 인증을 완료하지 않고 주문 조회 요청 으로 우회 발생

- ▶ 우회 예시



- ▶ 방지 예시



추가 인증 화면을 보여주는 것과 서버가 완료 여부를 확인하는 것은 별개의 작업이다

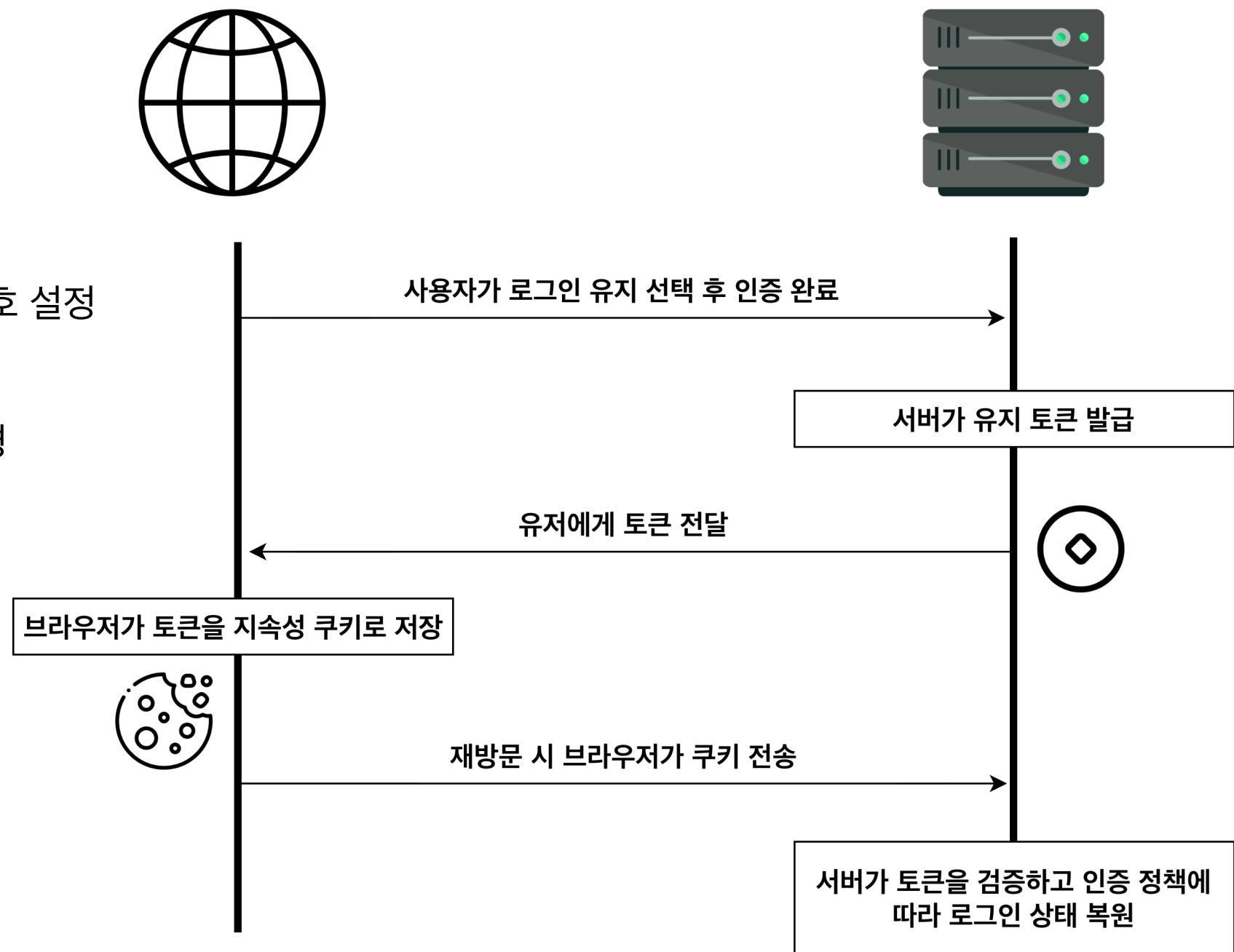
로그인 유지·비밀번호 재설정

▶ 로그인과 연결된 다른 기능

- 로그인 유지
 - 다시 방문해도 로그인 상태 유지
- 비밀번호 재설정
 - 비밀번호를 잊었을 때 새 비밀번호 설정
- 비밀번호 변경
 - 로그인한 사용자가 비밀번호 변경

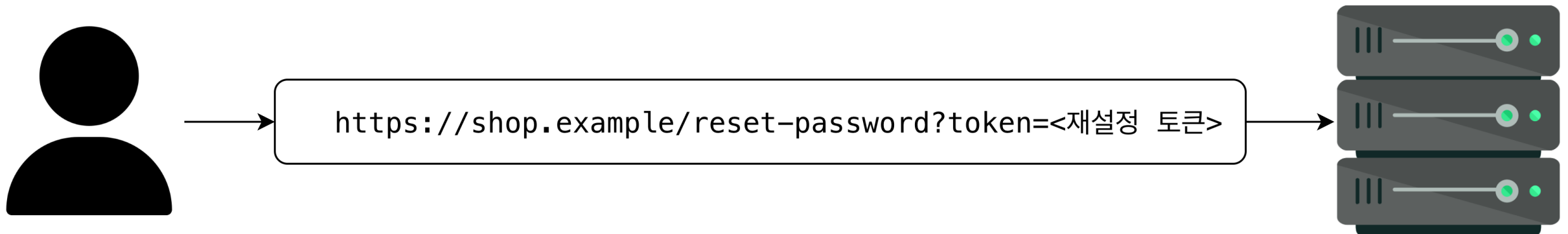
▶ 잘못된 예시

- 예측 가능한 쿠키 사용
- 단순 인코딩으로 암호화



비밀번호 재설정

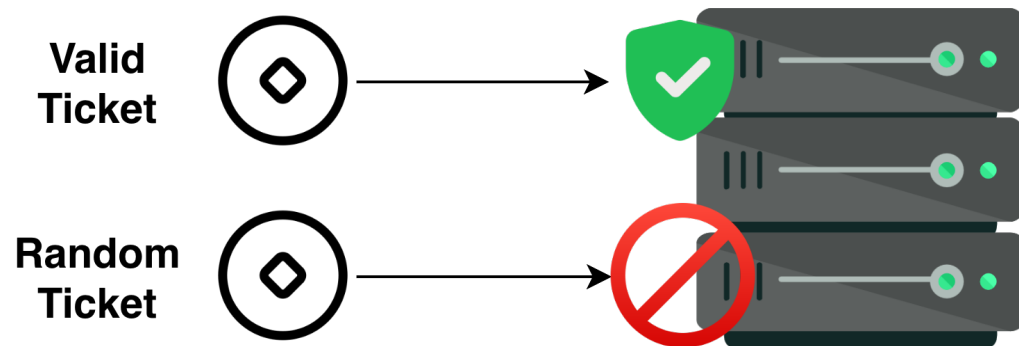
- ▶ 비밀번호 재설정의 정상 흐름
 1. 사용자가 비밀번호 재설정 요청
 2. 서버가 해당 계정에 연결된 재설정 토큰 생성
 3. 등록된 이메일로 재설정 링크 전송
 4. 사용자가 링크를 열고 새 비밀번호 제출
 5. 서버가 재설정 권한을 검증한 뒤 변경하고 토큰 무효화



비밀번호 재설정

- ▶ 재설정 요청에서 확인할 사항

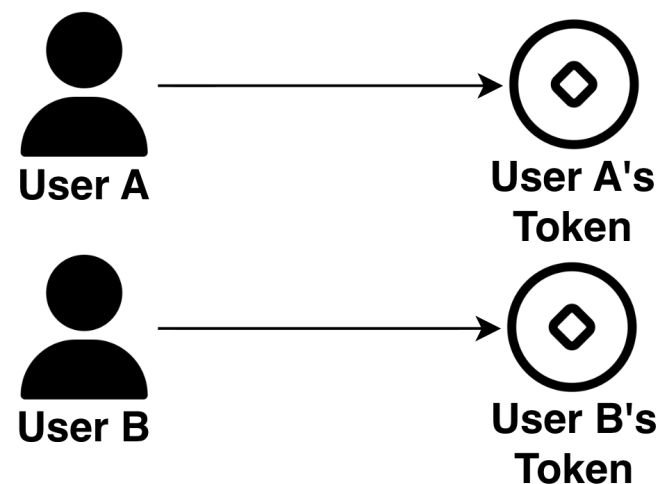
토큰 유효성



유효 시간



계정 연결



사용 여부



비밀번호 재설정

- ▶ 재설정 요청에서 확인할 사항



Authentication 1-day

Flask-HTTPAuth: CVE-2026-34531

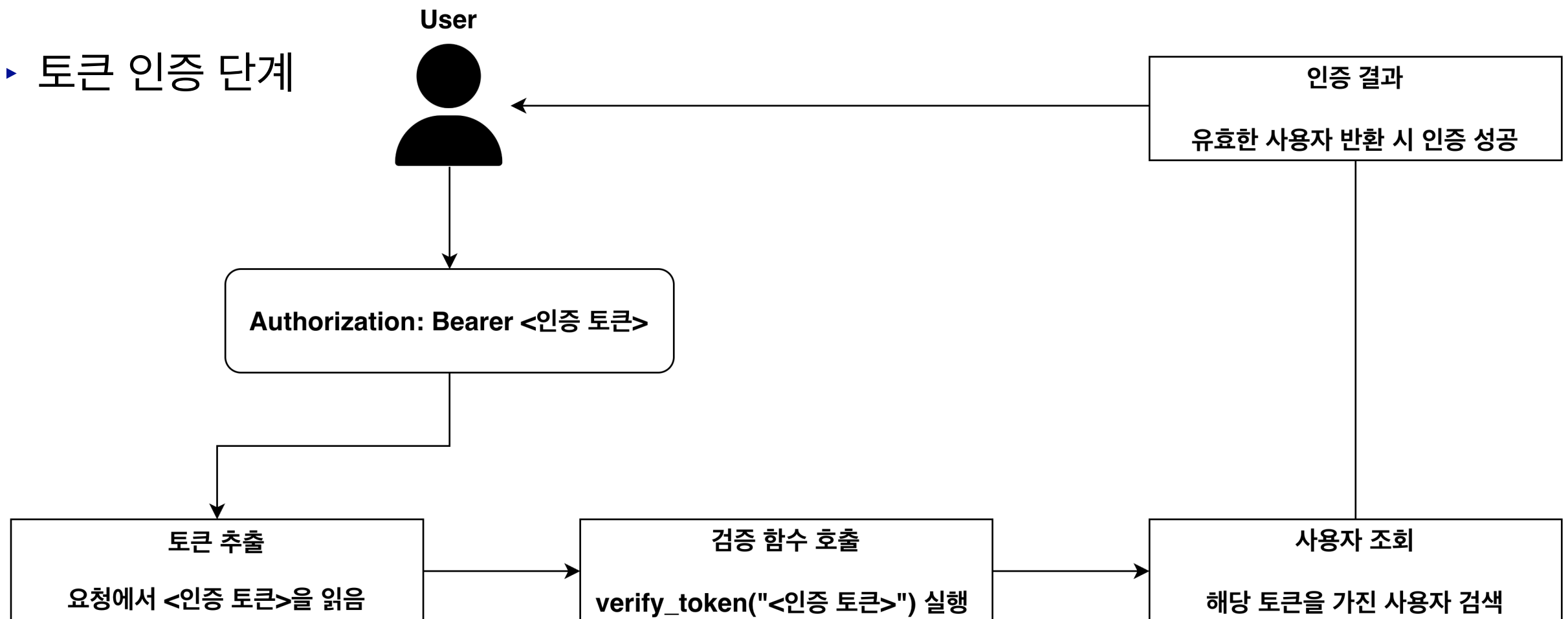
- ▶ Flask-HTTPAuth란?
 - Flask: Python으로 웹 어플리케이션을 만드는 프레임워크
 - Flask-HTTPAuth: 웹 경로에 HTTP 인증을 적용하는 확장
 - 요청에서 인증 정보를 읽고, 개발자가 등록한 검증 함수를 호출
- ▶ CVE-2026-34531
 - CVSS: 8.2 (High)
 - 취약점 유형: CWE-287 (Improper Authentication)
 - 영향 받는 소프트웨어: 버전 4.8.1 미만의 Flask-HTTPAuth, 이를 포함하는 IBM Storage 등
 - 원인 및 영향
 - 빈 토큰 처리 오류로 인해 권한이 없는 사용자가 인증을 통과할 수 있는 취약점

정상적인 토큰 인증

- ▶ 브라우저가 보내는 헤더 예시

Authorization: Bearer <인증 토큰>

- ▶ 토큰 인증 단계



토큰이 없는경우의 처리

- ▶ 취약 버전의 **HTTPTokenAuth.authenticate()** 는 다음과 같이 토큰을 가져온다.

```
token = getattr(auth, "token", "")  
  
if self.verify_token_callback:  
    return self.verify_token_callback(token)
```

- 토큰 속성이 없으면 기본값 "" 사용
- 검증 함수가 등록 되었는지 확인
- 토큰이 **verify_token("")** 호출

빈 문자열의 인증 우회

- ▶ 다음과 같은 DB가 있다고 가정

사용자	저장된 토큰
victim	빈 문자열 ""
minsu	"abc123"

- 검증 함수가 **token = ""** 인 사용자를 검색
- victim 사용자 정보가 반환될 수 있음
- 토큰을 제시하지 않은 요청이 victim으로 인증됨

조건: 토큰 인증 + DB 조회 방식의 검증 + 빈 문자열 토큰 사용자

수정 방안

- ▶ 수정 방안
 - 토큰이 없거나 비어 있으면 인증 실패
 - 사용자 조회 전에 토큰의 존재 확인
 - 토큰 미발급 상태는 DB에서 NULL로 관리
 - 애플리케이션 검증 함수에서도 빈 입력 거부

없는 자격 증명으로 사용자 검색을 시작하지 않는다.

실제 패치

▶ 실제 변경 부분

```
- token = getattr(auth, 'token', '')  
- if self.verify_token_callback:  
+ token = getattr(auth, 'token', None)  
+ if token and self.verify_token_callback:
```

- 변경점
 - "" 를 None 으로 변경 → 토큰이 없는 상태를 표현
 - if token 추가 → 토큰이 있어야 검증 함수 호출
 - 테스트 추가 → 빈 토큰 요청이 401로 실패하는지 확인
- 수정 버전: Flask-HTTPAuth 4.8.1

Jupyter Scheduler: CVE-2024-28188

▶ Jupyter Scheduler란?

- Jupyter: 브라우저에게 노트북과 코드를 다루는 플랫폼
- Jupyter Scheduler: 노트북 작업을 즉시 또는 예약 실행하는 확장
- 작업에 사용할 수 있는 실행 환경을 조회하는 API 제공

▶ CVE-2024-28188

- CVSS: 5.3 (Medium)
- 취약점 유형: CWE-287 (Improper Authentication), CWE-200 (Information Disclosure)
- 영향 받는 소프트웨어: Jupyter Scheduler 1.0.0~1.1.5, 1.2.0, 1.3.0~1.8.1, 2.0.0~2.5.1
- 원인 및 영향
 - 실행 환경 조회 API의 인증 검사 누락
 - 미인증 사용자가 서버의 Conda 환경 이름 목록을 조회할 수 있는 취약점

API 인증 확인

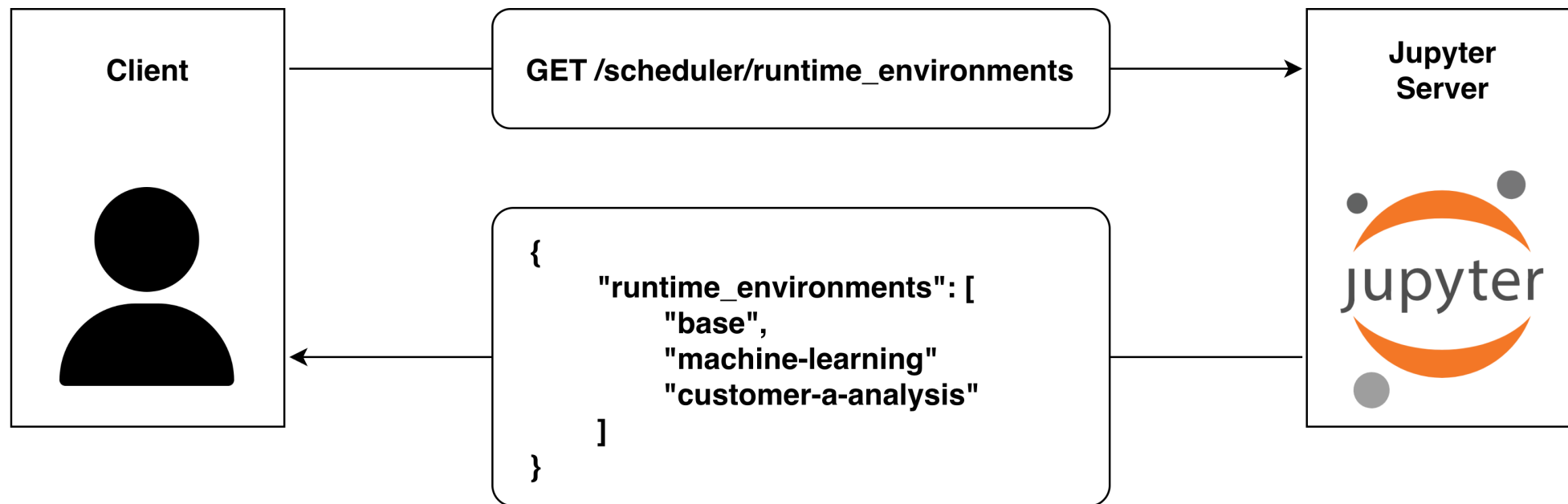
- ▶ 보호 대상 메서드의 예시

```
@authenticated  
async def get(self):  
    ...
```

- 핸들러: HTTP 요청을 처리하는 함수
- 데코레이터: 함수 실행에 추가 검사를 적용하는 문법
- @authenticated: 인증된 요청만 조회 함수 실행 허용
 - Jupyter가 토큰이나 세션으로 확인한 현재 사용자 정보를 바탕으로, 인증된 요청만 실제 함수에 전달.

환경 조회 API에 빠진 인증 검사

- 문제가 된 엔드포인트 /scheduler/runtime_environments



- 매서드 위에 @authenticated 누락
- 미인증 요청도 조회 로직에 도달
- 서버의 Conda 환경 이름 목록 반환

```
# RuntimeEnvironmentsHandler  
async def get(self):  
    ...
```

환경 이름 노출의 문제점

호출된 이름 예시

customer-a-analysis
production-forecast
python37-legacy

- 고객 및 프로젝트 이름: 내부 업무나 프로젝트 추측 가능
- 운영 목적: 서버와 작업의 용도 추측 가능
- 버전 표시: 사용중인 기술 구성 추측 가능

공격자는 환경 이름에 담긴 내부 정보를 통해 고객 관련 작업이나,
운영 환경의 존재를 추측할 수 있다.

수정 방향

- ▶ 보호할 API의 진입점에 인증 검사를 적용한다
 - 환경 조회 함수 실행 전에 인증 요구
 - 화면을 거치지 않는 직접 API 요청도 검사
 - 미인증 요청 차단과 정상 조회 기능을 함께 확인
- ▶ 확인할 요청 및 결과
 - 토큰 및 로그인 세션 없는 요청 → **환경 목록 제공 거부**
 - 유효한 인증 요청 → **기존 조회 기능 유지**

실제 패치

- ▶ RuntimeEnvironmentsHandler.get() 의 변경 부분

```
# RuntimeEnvironmentsHandler  
async def get(self):  
    ...
```



```
+ @authenticated  
async def get(self):  
    ...
```

- 조회 함수 앞에 인증 데코레이터 추가
- 핸들러 파일에 한 줄 추가
- 여러 유지보수 버전에 수정 반영
- 수정 버전
 - **>= 1.0.0, < 1.1.6** → **1.1.6**
 - **1.2.0** → **1.2.1**
 - **>= 1.3.0, < 1.8.2** → **1.8.2**
 - **>= 2.0.0, < 2.5.2** → **2.5.2**

Sentry: CVE-2025-22146

▶ Sentry란?

- 애플리케이션의 오류와 성능 정보를 확인하는 서비스
- 개발 조직이 사용하는 만큼 사용자 계정에 프로젝트와 오류 정보에 접근하는 권한이 연결

▶ CVE-2025-22146

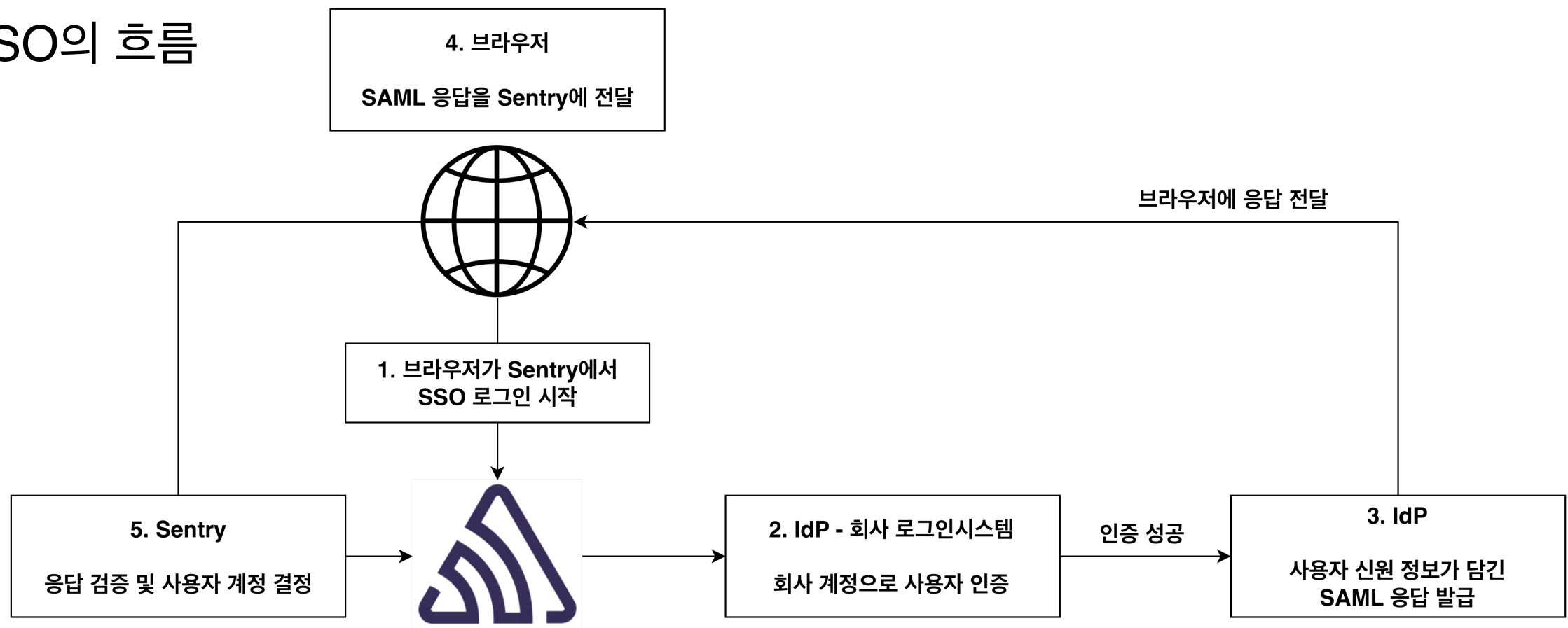
- CVSS: 9.1 (Critical)
- 취약점 유형: CWE-287 (Improper Authentication)
- 영향 받는 소프트웨어: 22.12.0 이상 25.1.0 미만의 Sentry
- 원인 및 영향
 - SAML 신원을 기존 계정을 연결할 때 현재 요청자와 연결 대상 계정의 동일성 검증 누락
 - 공격자가 자신의 외부 신원을 Victim 계정에 연결하고 계정을 탈취할 수 있는 취약점

외부 로그인 시스템을 이용하는 SSO

▶ SSO(Single Sign-On)

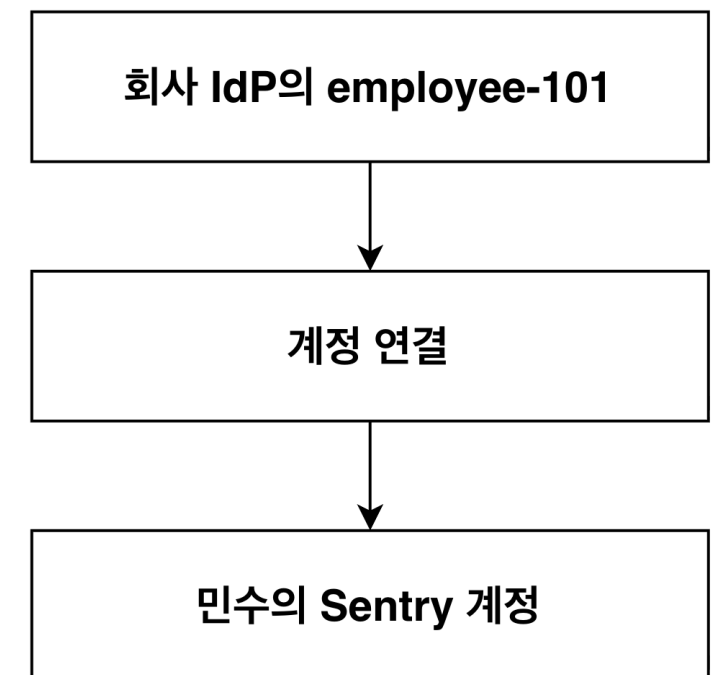
- 한 로그인 시스템의 인증을 여러 서비스에서 이용하는 방식
- IdP(Identify Provider): 사용자를 인증하고 신원 정보를 발급하는 시스템 → 회사 로그인 시스템
- SP(Service Provider): 신원 정보를 받아 로그인에 사용하는 서비스 → Sentry
- SAML(Security Assertion Markup Language): 신원 정보를 전달하는 표준 → SAML 응답 전달

▶ SSO의 흐름



외부 신원과 내부 계정의 연결

- ▶ 외부 로그인 신원과 Sentry 계정은 별도의 데이터이다.
 - 구분해야 할 객체
 - `self.request.user` → 현재 요청 세션의 사용자
 - `self.identity` → IdP가 전달한 외부 신원
 - `self.user` → 이 흐름에서 이메일로 찾은 연결 대상 계정
 - 계정 연결이 완료되면 이 외부신원으로 Sentry 로그인 가능
 - 계정 연결에서도 소유 여부를 확인해야 함
 - 이메일로 찾아낸 계정이 현재 요청자의 계정이라 판단 불가
 - `self.user` 및 `self.request.user`를 구분해야 함



취약점이 발생하는 상황

- ▶ 공격 조건
 - 공격자가 SAML IdP를 제어
 - 같은 Sentry 인스턴스에 공격자 조직과 피해자 계정 존재
 - 공격자가 피해자의 이메일 주소를 알고 있음
 - 여러 조직을 허용하는 환경



왜 취약한가

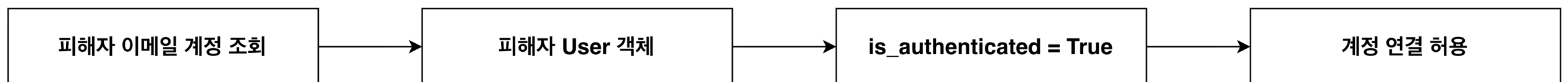
▶ 취약한 조건식

```
if op == "confirm" and self.user.is_authenticated or is_account_verified:
```

- op == “confirm” : 계정 연결 확인 요청
- self.user.is_authenticated : 연결 대상 사용자 객체의 속성
- is_account_verified : 별도 계정 확인 완료 여부

▶ 문제점

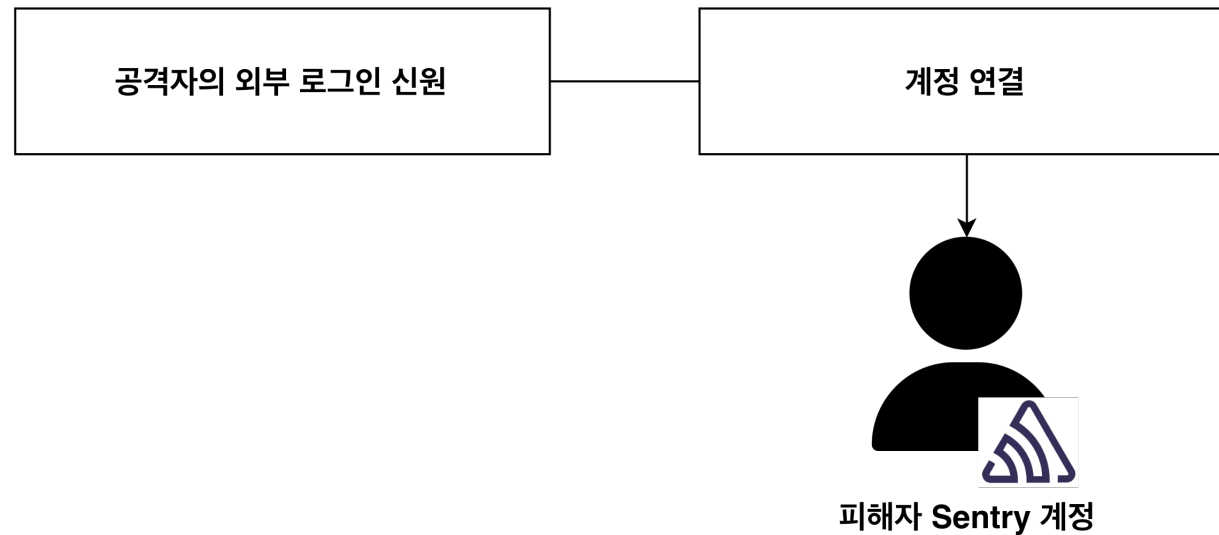
- 구체적인 Django User 객체의 is_authenticated는 항상 True
- 해당 속성은 유효한 세션이나 계정 소유 여부를 직접 검사하지 않음



핵심: 연결 대상 계정의 속성으로 현재 요청자의 소유 여부를 판단했다.

발생하는 문제 - 피해자 계정 탈취

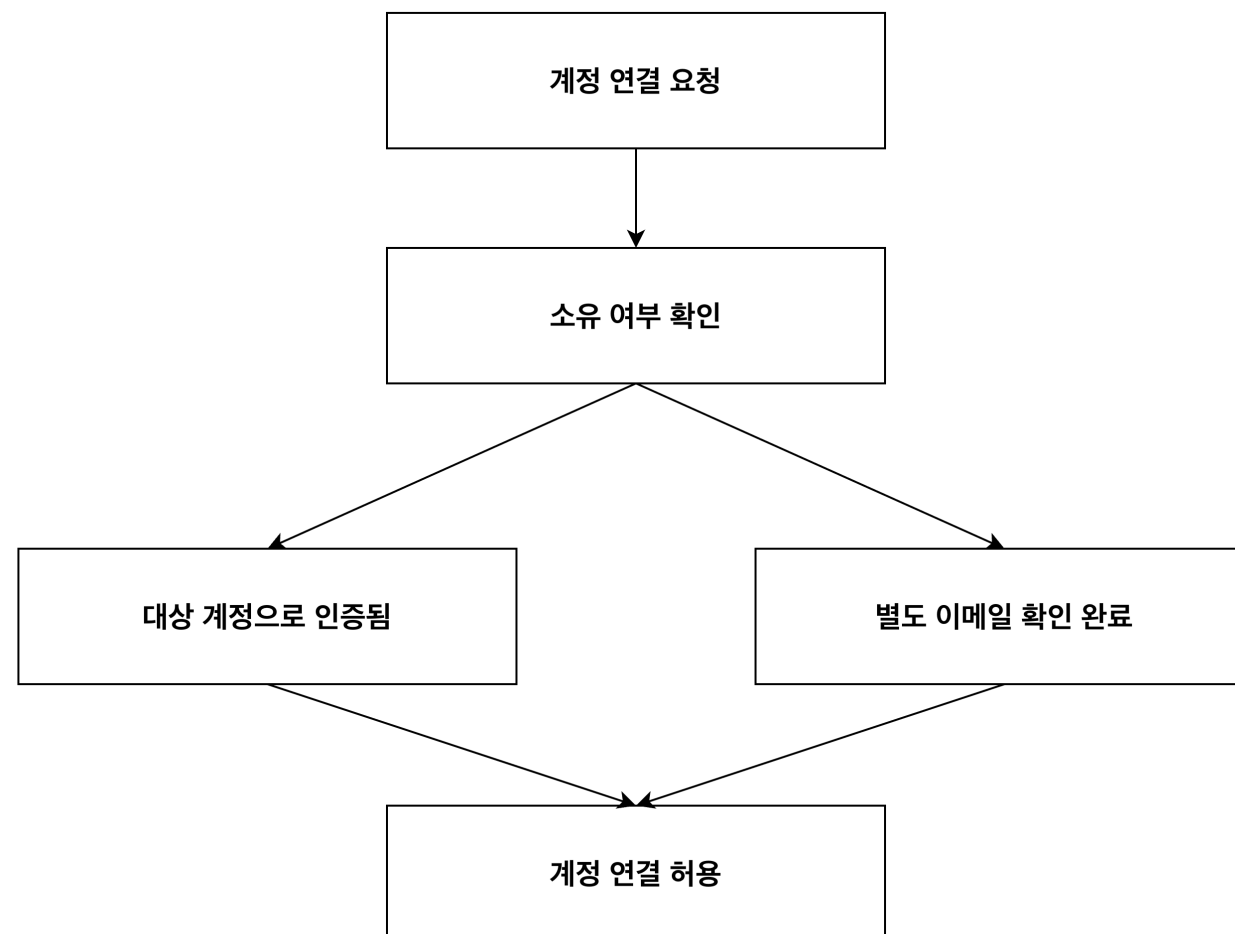
▶ 잘못된 연결



- 공격자의 외부 신원이 피해자 계정의 로그인 수단이 됨
- 피해자 권한으로 정보 조회 및 변경 가능
- 피해자 권한에 따라 프로젝트와 조직 설정에 영향

수정사항

- ▶ 패치 아이디어 - 연결 대상 계정의 소유 여부 확인
 - 계정 연결을 허용하려면 소유 여부를 입증해야 한다.
 - 해당 계정으로 인증된 세션 - 현재 요청자와 연결 대상 계정이 동일한가?
 - 별도 계정 확인 완료 - 대상 계정에 대한 일회성 이메일 확인을 완료했는가?
 - 즉, 외부 IdP가 전달한 이메일만으로 기존 계정 연결을 허용하지 않는다.



실제 패치

▶ 현재 사용자와 대상 사용자 비교

```
- if op == "confirm" and self.user.is_authenticated or is_account_verified:  
+ if op == "confirm" and (self.request.user.id == self.user.id) or is_account_verified:
```

▶ 변경 사항

- 대상 객체의 인증 속성 확인 → 현재 요청자 ID와 연결 대상 계정 ID 비교
 - self.user.is_authenticated → self.request.user.id == self.user.id
- 별도 계정 확인 경로 유지
- 계정 확인 전 연결이 생성되지 않는 테스트 추가
- 수정 버전: 25.1.0

계정 연결에서는 누구의 계정을 연결하는지 확인해야 한다.